

# Restaurant App MVP — Software Requirements Specification

Assignment 1 • Fall 2026 • Frontend Only • React Native / Expo

## 1. Introduction

This SRS defines the Restaurant App MVP for customers/diners and restaurant managers. The document covers a frontend-only implementation using React Native, local mock data, React hooks and client-side persistence.

### 1.1 Purpose

The purpose is to provide a mobile prototype where diners can browse a menu, search and sort dishes, manage a cart, apply a promo, reserve tables, place orders and track order status. Managers can review incoming orders and access menu-management controls.

### 1.2 Scope

In scope: authentication UI, customer/manager roles, menu browsing, category filtering, search, debounce, favourites, cart, promo codes, order summary, table reservation, order tracking, profile/theme, manager dashboard, mock data and AsyncStorage persistence. Out of scope: real payments, backend services, external APIs, real push notifications, production authentication and live restaurant inventory.

### 1.3 Definitions and Acronyms

SRS = Software Requirements Specification. MVP = Minimum Viable Product. UML = Unified Modeling Language. Hook = React function for state/lifecycle behaviour. Context = React API for shared state. Reducer = pure function that calculates the next state from state/action. Mock Data = local sample data used without a backend. FlatList = React Native performant list component. AsyncStorage = local key-value persistence.

## 2. Overall Description

The app serves two roles: Customer/Diner and Restaurant Manager. Customers focus on discovery, ordering and reservations. Managers focus on incoming orders, reservations and menu administration.

### 2.1 User Roles

Customer: sign up/login, browse/search/sort menu, favourite items, manage cart, apply promo, reserve a table, place an order, track status and manage profile/theme. Manager: login, see dashboard, change incoming order status, review reservations and access menu-management controls.

### 2.2 User Stories

1. As a customer, I want to browse daily specials, so that I can choose quickly. 2. As a customer, I want to search dishes, so that I can find food quickly. 3. As a customer, I want category filters, so that I can focus on one type of food. 4. As a customer, I want a cart, so that I can review my order before checkout. 5. As a customer, I want to reserve a table, so that I can arrive with a place ready. 6. As a customer, I want to apply a promo, so that I can see my discount. 7. As a customer, I want to track an order, so that I know its progress. 8. As a manager, I want to view incoming orders, so that I can process them. 9. As a manager, I want to update order status, so that customers see progress. 10. As a manager, I want menu availability controls, so that unavailable dishes are not ordered.

## 3. Functional Requirements

FR-01 The system shall validate login email and password. FR-02 The system shall validate signup password length and digit requirement. FR-03 The system shall authenticate against local mock users. FR-04 The system shall show at least fifteen menu items across four categories. FR-05 The system shall filter menu items by

category. FR-06 The system shall search menu items after debounce. FR-07 The system shall sort menu items by price or name. FR-08 The system shall show unavailable items as disabled. FR-09 The system shall add available items to the cart. FR-10 The system shall increment item quantities. FR-11 The system shall decrement and remove zero-quantity items. FR-12 The system shall store item-specific notes. FR-13 The system shall accept valid local promo codes. FR-14 The system shall calculate service charge, tax, discount and total. FR-15 The system shall offer hourly reservation slots. FR-16 The system shall prevent party sizes outside 1–12. FR-17 The system shall validate Pakistani mobile format. FR-18 The system shall create a local reservation after confirmation. FR-19 The system shall create an order with Pending status. FR-20 The system shall progress demo order status over time. FR-21 The system shall provide manager-only dashboard navigation. FR-22 The system shall let a manager advance incoming order status. FR-23 The system shall persist orders locally with AsyncStorage. FR-24 The system shall support light/dark theme switching.

## 4. Non-Functional Requirements

Usability: controls use clear labels, disabled states and confirmation feedback. Performance: menu uses FlatList and derived data uses useMemo; search is debounced. Responsiveness: layouts use flexible React Native dimensions and safe navigation patterns. Maintainability: code is divided into components, screens, context, reducers, hooks, data and navigation folders. Data handling: the prototype uses local JavaScript data and AsyncStorage only; no backend is required.

## 5. Client-Side Data Model

Users: id, fullName, email, password, role. Categories: category names. MenuItems: id, name, description, price, category, image, isSpecial, isAvailable. Tables: id, number, seats. Reservations: id, date, time, partySize, tableId, name, phone, status. Orders: id, items, total, type, status, timestamp.

## 6. UML Diagrams

The A1/UML folder contains five diagrams: Use Case, Class, Sequence, State Machine and Component. Each diagram models the corresponding actors, entities, order lifecycle and React Native architecture.

## 7. MVP Frontend Development

7.1 Login/Signup uses useState for controlled fields, errors, password visibility and submitting state. 7.2 Menu uses useState/useEffect for loading, categories and refresh. 7.3 Search uses useRef for input/list/debounce/render counter and useDebounce. 7.4 Profile uses AuthContext and ThemeContext. 7.5 Cart uses CartContext/useReducer. 7.6 Order Summary uses useMemo for totals and useCallback/React.memo in menu cards. 7.7 Reservation uses useReservation for date/time/table/contact logic. 7.8 Tracking uses useEffect timers and OrdersContext; Manager Dashboard is role-restricted.